

Explicação da Lógica Técnica - Sprint 2

Elaborado por: Desenvolvedores da Sprint 2

Destinado a: Willian Gomes Pessoa (Quality Manager)

Sprint: 2 - Cadastro do Domínio Principal (14/05 a 20/05/2026)

Visão geral dos módulos adicionados

A Sprint 2 expandiu a aplicação com dois novos blueprints ativos: `disciplinas` e `monitorias`. Ambos seguem o mesmo padrão da Sprint 1 (routes → service → repository), com queries SQL diretas e sem ORM.

...

browser

└─ Flask (app.py)

└─ auth/ ← Sprint 1

└─ usuarios/ ← Sprint 1

└─ disciplinas/ ← Sprint 2 (US06)

└─ routes.py (endpoints HTTP)

└─ service.py (regras de negócio)

└─ repository.py (queries SQL)

└─ monitorias/ ← Sprint 2 (US07, US08)

└─ routes.py

└─ service.py

└─ repository.py

...

US06 - Cadastro de disciplinas

Fluxo principal (admin cadastra disciplina):

1. Admin preenche código, nome e seleciona professor responsável
2. Backend valida: código não pode ser duplicado; professor_id deve pertencer a um usuário com papel `PROFESSOR`
3. Disciplina criada com status `ATIVA`
4. Admin pode editar, desativar/reactivar e matricular alunos na disciplina

Matrícula de alunos:

- Individual: admin seleciona um aluno pelo dropdown

- Em lote: admin cola lista de emails separados por vírgula ou quebra de linha; o sistema ignora emails não encontrados e reporta quantos foram adicionados

Endpoints:

...

GET/POST /disciplinas/ (admin only)

POST /disciplinas/<id>/editar (admin only)

POST /disciplinas/<id>/desativar (admin only)

POST /disciplinas/<id>/ativar (admin only)

POST /disciplinas/<id>/matricular (admin only)

GET /disciplinas/<id>/alunos (admin e professor da disciplina)

POST /disciplinas/<id>/alunos/adicionar (admin only)

POST /disciplinas/<id>/alunos/remover (admin only)

...

Pontos de atenção para o QM:

- Professor só vê alunos da disciplina dele na rota /alunos - acesso verificado por session["user_id"]
- A matrícula em lote silencia erros de email não encontrado - comportamento intencional para não bloquear importações parciais

US07 - Professor indica aluno como monitor

Fluxo de indicação:

1. Professor acessa `GET /monitorias/indicar`
2. Sistema exibe apenas as disciplinas do professor logado (filtradas por `professor\_id = session["user\_id"]`)
3. Professor seleciona disciplina e aluno e confirma
4. Backend valida: disciplina pertence ao professor (verificação server-side); aluno tem papel `ALUNO`
5. Vínculo criado com status `PENDENTE\_APROVACAO`
6. Professor vê o histórico das suas indicações na mesma tela

Endpoints:

...

GET/POST /monitorias/indicar (professor only)

...

Pontos de atenção para o QM:

- A lista de alunos exibe todos os alunos ativos do sistema, não só os matriculados na disciplina - limitação conhecida, escopo da Sprint 3
 - A validação de propriedade da disciplina é feita no backend, não apenas no frontend - um professor não consegue indicar para disciplina de outro mesmo forjando o form
-

US08 - Admin aprova ou rejeita indicação de monitor

Fluxo de aprovação:

1. Admin acessa `GET /monitorias/pendentes` - lista todas as indicações com status `PENDENTE\_APROVACAO`
2. Para aprovar: `POST /monitorias/<id>/aprovar`
 - Verifica se o aluno já possui monitoria ativa em qualquer disciplina (constraint de unicidade)
 - Se sim: rejeita com erro `ALUNO\_JA\_MONITOR`
 - Se não: atualiza status para `ATIVO`
3. Para rejeitar: `POST /monitorias/<id>/rejeitar`
 - Registra motivo informado pelo admin
 - Atualiza status para `REJEITADO`
 - Indicação sai da fila de pendentes

Endpoints:

...

GET /monitorias/pendentes (admin only)

POST /monitorias/<id>/aprovar (admin only)

POST /monitorias/<id>/rejeitar (admin only)

...

Decisão de design registrada em ADR-0006:

A validação de "aluno já monitor" é feita no momento da aprovação (não da indicação) para permitir que o professor indique sem precisar saber o estado atual de cada aluno. A responsabilidade de controle fica no admin.

Pontos de atenção para o QM:

- Indicação com status REJEITADO ou ATIVO não aparece mais na fila de pendentes
 - Um aluno pode ter indicações pendentes em múltiplas disciplinas simultaneamente - apenas a primeira aprovada é aceita; as demais falham com ALUNO_JA_MONITOR
-

US09 - Admin lista monitorias ativas

Fluxo:

A função `list_active_monitorias()` em `monitorias/repository.py` retorna todos os vínculos com `status = 'ATIVO'`, incluindo disciplina, monitor e professor. A listagem é exibida como tabela "Monitorias ativas" na tela de gestão de disciplinas do admin. Quando não há registros, exibe mensagem de vazio.

Ponto de atenção para o QM:

A implementação foi integrada via PR #32 junto com US07/US08, não em branch separada. A flag de integração foi resolvida após revisão do QM confirmar que o código estava presente e funcional na branch principal.

Migração de banco - Sprint 2

Duas alterações no schema em relação à Sprint 1:

```
```sql
ALTER TABLE disciplinas ADD COLUMN status ENUM('ATIVA','INATIVA') NOT NULL
DEFAULT 'ATIVA';

CREATE TABLE IF NOT EXISTS disciplina_alunos (
 id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
 disciplina_id BIGINT UNSIGNED NOT NULL,
 aluno_id BIGINT UNSIGNED NOT NULL,
 CONSTRAINT uq_disciplina_aluno UNIQUE (disciplina_id, aluno_id)
);
```
```

As migrations são aplicadas automaticamente na inicialização via `_apply_migrations()` em `connection.py`, de forma idempotente.

TT06 - Publicação no Railway

Problema: As POs precisam acessar o sistema sem depender do ambiente local de nenhum dev.

Solução: Publicação no Railway com detecção automática de Python e plugin MySQL nativo.

Como funciona:

O Railway detecta o projeto via `railpack.json` (provider Python) e executa o `Procfile`:

```
web: gunicorn --chdir backend "app:create_app()" --bind 0.0.0.0:$PORT
```

...

O plugin MySQL do Railway injeta variáveis com nomes diferentes dos locais (`MYSQLHOST`, `MYSQLPORT`, `MYSQLUSER`, `MYSQLPASSWORD`, `MYSQLDATABASE`). O `config.py` aceita ambos os formatos sem quebrar o ambiente local.

Na inicialização, `init\_db()` aplica schema, migrations e seed de admins automaticamente - sem scripts manuais no banco de produção.

Arquivos alterados: `Procfile`, `railpack.json`, `requirements.txt`, `backend/config.py`, `backend/db/connection.py`